

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG VOICEBOT TƯ VẤN DINH DƯỠNG THỜI GIAN THỰC DỰA TRÊN AI HỘI THOẠI

TRẦN QUANG MINH
MSSV: 20225047

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: Nguyễn Thị Thu Trang

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

TÓM TẮT NỘI DUNG ĐỒ ÁN

Tư vấn dinh dưỡng qua giọng nói có ý nghĩa thực tiễn trong các hệ thống chăm sóc sức khỏe thông minh, tổng đài tự động và trợ lý cá nhân. So với chatbot văn bản, voicebot cho phép người dùng tương tác tự nhiên và thuận tiện hơn trong các tình huống khó sử dụng bàn phím hoặc màn hình. Tuy nhiên, hệ thống phải đồng thời giải quyết các thách thức về độ chính xác của nhận dạng giọng nói tiếng Việt, chất lượng truy xuất tri thức, độ tin cậy của câu trả lời và độ trễ tích lũy qua nhiều bước xử lý.

Đồ án xây dựng một hệ thống voicebot tư vấn dinh dưỡng tiếng Việt theo pipeline mô-đun gồm nhận dạng giọng nói, xử lý hội thoại dựa trên Retrieval-Augmented Generation và tổng hợp giọng nói. Kho tri thức được xây dựng từ 1060 bài viết dinh dưỡng thuộc các nguồn có tính chuyên môn. Câu hỏi sau nhận dạng được chuẩn hóa và mở rộng bằng từ điển thuật ngữ dinh dưỡng, sau đó được truy xuất trên Qdrant bằng mô hình embedding tiếng Việt. Các văn bản có mức độ tương đồng cao được đánh giá lại bằng mô hình reranker để lựa chọn một tập văn bản nhỏ hơn và phù hợp hơn làm ngữ cảnh cho mô hình ngôn ngữ. Hệ thống đồng thời sử dụng guardrail dựa trên luật và kiểm tra mức đủ tối thiểu của ngữ cảnh nhằm hạn chế câu trả lời ngoài phạm vi hoặc thiếu căn cứ.

Để giảm thời gian phản hồi, đồ án tổ chức cơ chế streaming song song giữa mô hình ngôn ngữ và mô-đun tổng hợp giọng nói. Văn bản được chia tại các vị trí ngắt tương đối tự nhiên và chuyển sang tổng hợp âm thanh ngay khi đủ điều kiện. Trên tập đánh giá RAG, cấu hình lấy 15 đoạn ứng viên và chọn 5 đoạn đạt faithfulness 0,7723. Trong thí nghiệm độ trễ, cấu hình chia văn bản ở ngưỡng 40 ký tự làm giảm trung vị Time-to-First-Audio từ khoảng 1523 ms xuống 747 ms. Kết quả cho thấy giải pháp có khả năng cân bằng giữa chất lượng câu trả lời và độ trễ

cảm nhận trong phạm vi tư vấn dinh dưỡng phổ biến.

Sinh viên thực hiện

Trần Quang Minh

MỤC LỤC

TÓM TẮT NỘI DUNG ĐỒ ÁN	i
1 GIỚI THIỆU BÀI TOÁN	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu và phạm vi	4
1.3 Định hướng giải pháp	6
1.4 Cấu trúc đồ án	8
2 CƠ SỞ LÝ THUYẾT	10
2.1 Tổng quan pipeline hội thoại bằng giọng nói	10
2.2 Nhận dạng giọng nói tự động	10
2.2.1 Bài toán và biểu diễn tín hiệu	10
2.2.2 RNN-Transducer	11
2.2.3 Conformer và Zipformer	12
2.2.4 Các chỉ số đánh giá ASR	13
2.3 Mô hình ngôn ngữ lớn	14
2.4 Retrieval-Augmented Generation	15
2.4.1 Chuẩn hóa và mở rộng truy vấn	15
2.5 Truy xuất ngữ nghĩa và reranking	16
2.5.1 Embedding và bi-encoder	16
2.5.2 Vector database và HNSW	16
2.5.3 Cross-encoder và truy xuất hai bước	17

2.6	Tổng hợp giọng nói	17
2.6.1	Chuẩn hóa và biểu diễn văn bản	18
2.6.2	Neural audio codec	18
2.6.3	Mô hình ngôn ngữ sinh token tiếng nói	19
2.6.4	Sinh âm thanh theo luồng	19
2.7	Streaming và xử lý bất đồng bộ	20
2.8	Các phương pháp đánh giá	21
2.8.1	Đánh giá hệ thống RAG	21
2.8.2	Đánh giá độ trễ	21
3	GIẢI PHÁP ĐỀ XUẤT	23
3.1	Kiến trúc tổng quan	23
3.2	Mô-đun xử lý tiếng nói	24
3.2.1	Nhận dạng giọng nói	24
3.2.2	Tổng hợp giọng nói	25
3.3	Mô-đun xử lý hội thoại	25
3.3.1	Tiền xử lý truy vấn	25
3.3.2	Retrieval	26
3.3.3	Augmentation	27
3.3.4	Generation	27
3.4	Chiến thuật streaming	28
4	PHÁT TRIỂN HỆ THỐNG, THỰC NGHIỆM VÀ ĐÁNH GIÁ	30
4.1	Xây dựng kho tri thức	30
4.2	Phát triển hệ thống	31

4.2.1	Thiết kế hệ thống	31
4.2.2	Xây dựng hệ thống	33
4.3	Thiết lập thực nghiệm	34
4.4	Đánh giá chất lượng RAG	35
4.5	Đánh giá độ trễ	38
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
5.1	Kết luận	41
5.2	Hạn chế	42
5.3	Hướng phát triển	43

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1 Đặt vấn đề

Hệ thống hội thoại hỏi đáp tư vấn cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận lại thông tin hỗ trợ trong một phạm vi xác định. Khác với công cụ tìm kiếm chỉ trả về danh sách tài liệu, hệ thống hội thoại cần xác định nội dung người dùng quan tâm, khai thác nguồn tri thức phù hợp, duy trì ngữ cảnh giữa các lượt trao đổi và diễn đạt câu trả lời theo cách dễ hiểu. Tùy theo kênh tương tác, hệ thống có thể hoạt động dưới dạng chatbot văn bản hoặc voicebot. Chẳng hạn, cùng với câu hỏi “người bị tiểu đường nên hạn chế thực phẩm nào”, chatbot nhận văn bản được nhập từ bàn phím và hiển thị câu trả lời trên màn hình, trong khi voicebot tiếp nhận câu hỏi bằng tiếng nói và phản hồi lại bằng âm thanh.

Các hệ thống hỏi đáp có thể được ứng dụng trong chăm sóc khách hàng, giáo dục, tổng đài tự động và hỗ trợ tra cứu thông tin theo từng chủ đề. Ý nghĩa thực tiễn của chúng nằm ở khả năng tạo thêm một kênh tiếp cận thông tin, hỗ trợ giải đáp các câu hỏi thường gặp, phục vụ ngoài giờ hành chính và giảm thao tác tìm kiếm thủ công. Đối với những nội dung có phạm vi rõ ràng, hệ thống còn có thể sử dụng một kho tri thức được xây dựng trước để lựa chọn thông tin phù hợp thay vì yêu cầu người dùng tự đọc và tổng hợp từ nhiều tài liệu khác nhau.

Chatbot văn bản là hình thức phổ biến vì dễ tích hợp vào website, ứng dụng di động và nền tảng nhắn tin. Người dùng có thể đọc lại, lưu hoặc sao chép câu trả lời nên kênh này phù hợp với những nội dung dài hoặc cần xem xét chi tiết. Tuy nhiên, chatbot yêu cầu người dùng nhìn màn hình và thực hiện thao tác nhập liệu. Việc nhập câu hỏi dài có thể bất tiện khi người dùng đang di chuyển, đang thực hiện công việc khác, có

thị lực kém hoặc gặp khó khăn khi sử dụng thiết bị. Khả năng tiếp nhận công nghệ của người cao tuổi cũng chịu ảnh hưởng bởi năng lực số, khả năng vận động và mức độ dễ sử dụng của hệ thống. Do đó, kênh văn bản phù hợp khi người dùng có thể chủ động thao tác trên màn hình, nhưng chưa đáp ứng tốt những tình huống cần tương tác nhanh và không thể sử dụng tay để tương tác như khi lái xe.

Voicebot được sử dụng để bổ sung kênh tương tác bằng tiếng nói. Một voicebot thường gồm mô-đun nhận dạng giọng nói để chuyển âm thanh thành văn bản, mô-đun xử lý hội thoại để phân tích câu hỏi và tạo câu trả lời, mô-đun tổng hợp giọng nói để chuyển văn bản thành âm thanh, cùng thành phần điều phối luồng dữ liệu giữa các mô-đun. Kênh thoại có thể được ứng dụng trong tổng đài tự động, ứng dụng trợ lý giọng nói, khi người dùng đang lái xe, không thuận tiện sử dụng tay hoặc gặp khó khăn khi đọc nội dung trên màn hình. Vì lời nói là một phương thức giao tiếp tự nhiên, voicebot giúp giảm thao tác nhập liệu và cho phép người dùng tiếp cận hệ thống trong nhiều hoàn cảnh hơn. Các hệ thống hội thoại bằng văn bản và tiếng nói cũng được sử dụng như một kênh hỗ trợ cung cấp thông tin trong lĩnh vực chăm sóc sức khỏe, mặc dù hiệu quả và độ an toàn vẫn cần được đánh giá thận trọng.

Tuy nhiên, kênh thoại tạo ra nhiều thách thức kỹ thuật hơn kênh văn bản. Về độ chính xác, dữ liệu phải đi qua nhiều bước gồm nhận dạng giọng nói, xử lý câu hỏi, truy xuất tri thức, sinh câu trả lời và tổng hợp giọng nói. Sai số tại một bước có thể truyền sang các bước tiếp theo; chẳng hạn, nhận dạng sai một thuật ngữ có thể làm thay đổi truy vấn, dẫn tới lựa chọn sai tài liệu và tạo câu trả lời không đúng trọng tâm. Về độ trễ, thời gian phản hồi là tổng thời gian xử lý của nhiều mô-đun. Nếu toàn bộ pipeline được thực hiện tuần tự, người dùng phải chờ nhận dạng xong câu hỏi, truy xuất tài liệu, sinh hoàn chỉnh câu trả lời và tổng hợp toàn bộ âm thanh trước khi nghe được phản hồi. Khoảng chờ dài làm

giảm tính tự nhiên và có thể gây gián đoạn mạch hội thoại. Ngoài ra, câu trả lời đầu ra cần ngắn gọn, dễ nghe và được phân đoạn tại các vị trí phù hợp để hạn chế ngắt nhịp khi tổng hợp giọng nói.

Trong chủ đề dinh dưỡng, người dùng thường cần tra cứu những thông tin gần với sinh hoạt hằng ngày như lựa chọn nhóm thực phẩm, bổ sung vitamin và khoáng chất, kiểm soát cân nặng, dinh dưỡng cho phụ nữ mang thai, trẻ em, người cao tuổi hoặc chế độ ăn khi có một số bệnh nền phổ biến. Chế độ ăn lành mạnh cần bảo đảm tính đầy đủ, cân bằng và đa dạng. Tuy nhiên, thông tin dinh dưỡng trên Internet thường phân tán, khác nhau về mức độ chuyên môn và không phải lúc nào cũng phù hợp với câu hỏi cụ thể. Bài toán vì vậy cần một kho tri thức từ các nguồn được lựa chọn để câu trả lời có căn cứ và phù hợp hơn với chủ đề.

Dinh dưỡng có liên quan đến sức khỏe nên nội dung trả lời cần được kiểm soát thận trọng, nhưng điều đó không đồng nghĩa hệ thống thực hiện tư vấn y khoa. Hệ thống trong đề án chỉ cung cấp thông tin dinh dưỡng phổ thông dựa trên kho tri thức đã xây dựng, không chẩn đoán bệnh, kê đơn hoặc thay đổi phác đồ điều trị. Mô hình ngôn ngữ có thể tạo ra thông tin nghe hợp lý nhưng thiếu căn cứ; trong khi đó, lỗi nhận dạng tiếng nói hoặc truy xuất sai tài liệu còn có thể làm tăng sai lệch của câu trả lời. Những câu hỏi có dấu hiệu khẩn cấp, yêu cầu chẩn đoán hoặc vượt khỏi phạm vi dinh dưỡng cần được chuyển sang phản hồi thận trọng và khuyến nghị người dùng tìm đến chuyên gia phù hợp.

Từ các vấn đề trên, đề án xây dựng hệ thống hỏi đáp dinh dưỡng tiếng Việt bằng giọng nói, sử dụng kho tri thức để tạo câu trả lời có căn cứ và tổ chức cơ chế streaming giữa mô-đun xử lý hội thoại với tổng hợp giọng nói nhằm tối ưu trải nghiệm người dùng.

1.2 Mục tiêu và phạm vi

Phạm vi của đề án được giới hạn trong bài toán tư vấn dinh dưỡng bằng tiếng Việt qua kênh thoại. Về ngôn ngữ, hệ thống tập trung xử lý tiếng Việt ở cả đầu vào giọng nói, câu hỏi văn bản trung gian và câu trả lời đầu ra. Về chủ đề, hệ thống chỉ xử lý các câu hỏi dinh dưỡng phổ biến như lựa chọn thực phẩm, chế độ ăn hằng ngày, vi chất, nhóm thực phẩm, kiểm soát cân nặng, dinh dưỡng cho phụ nữ mang thai, trẻ em, người cao tuổi và một số tình trạng sức khỏe thường gặp. Đề án không đặt mục tiêu thay thế bác sĩ, chuyên gia dinh dưỡng hoặc hệ thống chẩn đoán y khoa.

Về kênh tương tác, đề án tập trung vào voicebot thay vì chatbot văn bản thuần túy. Người dùng đặt câu hỏi bằng giọng nói, hệ thống nhận dạng thành văn bản, truy xuất tri thức, sinh câu trả lời và phát lại bằng giọng nói. Giao diện văn bản chỉ đóng vai trò hỗ trợ hiển thị nội dung nhận dạng và trạng thái xử lý; mục tiêu chính vẫn là trải nghiệm hội thoại bằng giọng nói với độ trễ thấp.

Về yêu cầu thời gian thực, hệ thống xử lý theo từng lượt hội thoại và ưu tiên giảm thời gian từ khi người dùng kết thúc câu hỏi đến khi nghe được âm thanh phản hồi đầu tiên. Chỉ số chính được sử dụng để đánh giá yêu cầu này là Time-to-First-Audio.

Về dữ liệu, kho tri thức được xây dựng từ 1060 bài viết dinh dưỡng thu thập từ Báo Sức khỏe và Đời sống, Viện Dinh dưỡng Quốc gia và Bệnh viện Đa khoa Quốc tế Thu Cúc. Dữ liệu được làm sạch và phân đoạn để phục vụ quá trình truy xuất tri thức. Nội dung trả lời của hệ thống được giới hạn trong phạm vi thông tin do kho tri thức này cung cấp.

Phạm vi nghiên cứu không bao gồm xây dựng mô hình ASR, LLM hoặc TTS từ đầu; đề án lựa chọn và tích hợp các mô hình có sẵn, sau đó tập trung vào tổ chức pipeline, xây dựng kho tri thức, cải thiện truy xuất

và tối ưu luồng phản hồi. Hệ thống chưa hướng tới tư vấn cá nhân hóa dựa trên hồ sơ bệnh án, không xử lý dữ liệu khám chữa bệnh và không đánh giá hiệu quả lâm sàng. Kết quả của hệ thống chỉ có vai trò cung cấp thông tin tham khảo ban đầu trong phạm vi kho tri thức đã xây dựng.

Các mục tiêu cụ thể của đề án gồm:

- 1) Xây dựng pipeline tương tác hai chiều bằng tiếng nói gồm ASR, mô-đun xử lý hội thoại và TTS, có khả năng xử lý một lượt hỏi đáp dinh dưỡng từ giọng nói đầu vào đến giọng nói đầu ra.
- 2) Tối ưu độ trễ phản hồi, trong đó chỉ số trọng tâm là Time-to-First-Audio. Hệ thống được đánh giá theo p50 và p90; ngưỡng mục tiêu trong môi trường thử nghiệm là TTFA p50 không vượt quá 1,5 giây và TTFA p90 không vượt quá 2 giây.
- 3) So sánh pipeline tuần tự với pipeline streaming LLM–TTS, đồng thời đánh giá ảnh hưởng của kích thước đoạn văn bản đến Time-to-First-Audio và tổng thời gian phản hồi; phân tích trade-off giữa việc giảm độ trễ với nguy cơ ngắt nhịp khi đoạn văn bản quá ngắn.
- 4) Nâng cao mức độ bám ngữ cảnh và độ phủ truy xuất bằng RAG trên kho tri thức dinh dưỡng, kết hợp mở rộng truy vấn, truy xuất vector, reranking và prompt ràng buộc theo ngữ cảnh. Trong phạm vi tập đánh giá, hệ thống đạt mục tiêu faithfulness đạt tối thiểu 0,70 và context recall đạt tối thiểu 0,60.
- 5) Đánh giá khả năng xử lý của ASR bằng Real-Time Factor, với mục tiêu RTF nhỏ hơn 1 để thời gian nhận dạng không vượt quá thời lượng audio đầu vào; đồng thời đánh giá câu trả lời theo mức đúng trọng tâm, bám sát ngữ cảnh và hạn chế suy diễn ngoài tài liệu.
- 6) Thiết kế hệ thống theo pipeline mô-đun để dễ triển khai, kiểm thử và mở rộng, trong đó Gateway có nhiệm vụ điều phối luồng dữ liệu,

hàng đợi văn bản–âm thanh và trạng thái phiên hội thoại.

Các câu hỏi đánh giá xoay quanh các chủ đề như đái tháo đường, phụ nữ mang thai, trẻ biếng ăn, vitamin, khoáng chất, chế độ ăn giảm cân, omega-3, canxi và các nhóm thực phẩm thường gặp. Hệ thống không tự đưa ra chẩn đoán bệnh, không kê đơn thuốc và không thay đổi phác đồ điều trị. Với các câu hỏi vượt phạm vi hoặc có dấu hiệu rủi ro y tế, hệ thống cần từ chối mềm, khuyến nghị người dùng gặp chuyên gia phù hợp hoặc yêu cầu đặt lại câu hỏi trong phạm vi dinh dưỡng.

Như vậy, đồ án không chỉ đánh giá hệ thống theo khả năng sinh câu trả lời đúng trọng tâm, mà còn theo khả năng vận hành trong tương tác giọng nói có độ trễ thấp. Hai nhóm chỉ số chính là: nhóm độ trễ gồm Time-to-First-Audio, tổng thời gian phản hồi, Real-Time Factor và độ trễ từng mô-đun; nhóm chất lượng gồm mức liên quan của tài liệu truy xuất, độ phủ ngữ cảnh và mức độ câu trả lời bám vào ngữ cảnh. Ảnh hưởng của kích thước đoạn văn bản đến nhịp đọc được phân tích như một trade-off thiết kế, chưa được xem là kết quả đánh giá chủ quan chính thức.

1.3 Định hướng giải pháp

Định hướng giải pháp của đồ án là xây dựng một pipeline tương tác hai chiều bằng tiếng nói cho tư vấn dinh dưỡng tiếng Việt. Audio được truyền từ trình duyệt tới hệ thống theo các khung ngắn để phục vụ phát hiện tiếng nói và xác định kết thúc lượt nói. Sau đó, ASR tạo transcript cuối cùng cho lượt hội thoại; mô-đun xử lý hội thoại kiểm tra phạm vi câu hỏi, chuẩn hóa truy vấn, truy xuất tri thức và sinh câu trả lời; TTS chuyển câu trả lời thành âm thanh để phát lại cho người dùng. Gateway đóng vai trò điều phối phiên hội thoại, liên kết các mô-đun và bảo đảm dữ liệu của các lượt hỏi đáp được xử lý theo đúng phiên.

Để hạn chế việc mô hình ngôn ngữ trả lời chỉ dựa trên tri thức tham số, hệ thống sử dụng Retrieval-Augmented Generation. Kho bài viết định dưỡng sau khi làm sạch được phân đoạn, mã hóa thành vector embedding và lưu trong vector database. Với mỗi câu hỏi, hệ thống thực hiện truy xuất hai bước: lấy một tập văn bản ứng viên bằng tìm kiếm vector, sau đó dùng mô hình reranker để chọn một tập nhỏ hơn làm ngữ cảnh cho LLM. Trước bước sinh câu trả lời, lớp kiểm soát đầu vào dựa trên luật được sử dụng để phát hiện một số câu hỏi ngoài phạm vi hoặc có dấu hiệu rủi ro; kết quả truy xuất cũng được kiểm tra ở mức tối thiểu nhằm tránh trả lời chặc chán khi không có đủ ngữ cảnh.

Đối với kênh thoại, độ trễ là vấn đề trung tâm của giải pháp. Nếu hệ thống xử lý tuần tự, người dùng phải chờ nhận dạng giọng nói, truy xuất tài liệu, sinh toàn bộ câu trả lời và tổng hợp xong toàn bộ âm thanh rồi mới nghe được phản hồi. Vì vậy, đề án định hướng tổ chức quá trình trả lời theo streaming: khi LLM sinh được một phần văn bản đủ điều kiện, phần văn bản đó được chuyển sớm sang TTS thay vì chờ toàn bộ câu trả lời hoàn tất. Chiến thuật phân đoạn văn bản dựa trên dấu câu và ranh giới từ được sử dụng để chia văn bản tại các vị trí tương đối tự nhiên, nhằm giảm Time-to-First-Audio nhưng vẫn hạn chế việc TTS đọc bị ngắt nhịp.

Về kiến trúc triển khai, các chức năng được tách thành các mô-đun Web Client, Gateway, ASR, xử lý hội thoại, TTS, Qdrant và Redis. Cách tổ chức này cho phép đo thời gian từng giai đoạn, thay thế mô hình độc lập và cô lập lỗi tốt hơn so với đặt toàn bộ pipeline trong một tiến trình. Các mô-đun phục vụ chung một luồng nghiệp vụ và được Gateway điều phối tập trung.

Để hiện thực định hướng trên, đề án kết hợp các kỹ thuật nhận dạng và tổng hợp giọng nói, mô hình ngôn ngữ lớn, Retrieval-Augmented Generation, truy xuất vector, reranking và xử lý dữ liệu dạng streaming. Các thành phần được lựa chọn và tổ chức nhằm bảo đảm câu trả lời dựa

trên kho tri thức dinh dưỡng, đồng thời giảm thời gian chờ trước khi người dùng nghe được âm thanh phản hồi đầu tiên. Hiệu quả của giải pháp được xem xét trên cả chất lượng truy xuất, mức độ câu trả lời bám sát ngữ cảnh và độ trễ của pipeline.

1.4 Cấu trúc đồ án

Phần còn lại của báo cáo được tổ chức theo mạch từ cơ sở lý thuyết, giải pháp kỹ thuật, quá trình hiện thực đến thực nghiệm và kết luận. Chương 1 giới thiệu bối cảnh tiếp cận thông tin dinh dưỡng, phân tích sự khác biệt giữa chatbot văn bản và voicebot, phát biểu bài toán, xác định mục tiêu và giới hạn phạm vi nghiên cứu.

Chương 2 trình bày cơ sở lý thuyết liên quan đến nhận dạng giọng nói, mô hình ngôn ngữ, RAG, vector database, embedding, reranking, RAGAS và tổng hợp giọng nói. Các nội dung này cung cấp nền tảng để giải thích vì sao hệ thống cần kết hợp kho tri thức bên ngoài với LLM, cũng như vì sao độ trễ trở thành vấn đề quan trọng trong bài toán voicebot.

Chương 3 mô tả giải pháp đề xuất, bao gồm kiến trúc tổng quan, các mô-đun xử lý tiếng nói, mô-đun xử lý hội thoại, quy trình RAG hai tầng và chiến thuật streaming. Đây là chương làm rõ cách hệ thống tổ chức pipeline tương tác hai chiều bằng tiếng nói, cách câu trả lời được sinh dựa trên kho tri thức, và cách streaming song song giữa LLM với TTS được sử dụng để tối ưu Time-to-First-Audio.

Chương 4 trình bày quá trình phát triển hệ thống, xây dựng kho tri thức và thiết kế đánh giá thực nghiệm. Phần đánh giá chất lượng sử dụng các chỉ số RAGAS để so sánh các cấu hình truy xuất. Phần đánh giá độ trễ tập trung nghiên cứu ảnh hưởng của kích thước đoạn văn bản chuyển sang TTS đối với Time-to-First-Audio và tổng thời gian phản hồi, đồng

thời thảo luận nguy cơ ngắt nhịp khi đoạn văn bản quá ngắn.

Chương 5 tổng kết kết quả đạt được và đối chiếu lại các đóng góp chính của đề án. Các đóng góp này gồm pipeline streaming song song tối ưu hóa thời gian phản hồi âm thanh đầu tiên, quy trình RAG hai tầng cho tiếng Việt chuyên ngành dinh dưỡng và kết quả thực nghiệm về ảnh hưởng của kích thước đoạn văn bản đối với độ trễ phản hồi. Chương này cũng nêu các hạn chế còn tồn tại và định hướng phát triển tiếp theo.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan pipeline hội thoại bằng giọng nói

Hệ thống hội thoại bằng giọng nói thực hiện hai phép chuyển đổi chính: chuyển tiếng nói của người dùng thành biểu diễn văn bản và chuyển câu trả lời văn bản trở lại thành âm thanh. Giữa hai phép chuyển đổi này là mô-đun xử lý hội thoại, có nhiệm vụ phân tích câu hỏi, khai thác nguồn tri thức và tạo nội dung phản hồi. Pipeline tổng quát có thể biểu diễn như sau:

Tiếng nói $\rightarrow$ ASR $\rightarrow$ Xử lý hội thoại $\rightarrow$ TTS $\rightarrow$ Âm thanh phản hồi.
(2.1)

Khác với chatbot văn bản, chất lượng của voicebot phụ thuộc đồng thời vào nhiều mô-đun. Một lỗi nhận dạng ở đầu vào có thể làm thay đổi ý nghĩa truy vấn, dẫn tới truy xuất sai tài liệu và sinh câu trả lời không phù hợp. Tương tự, một câu trả lời đúng về nội dung vẫn có thể tạo trải nghiệm kém nếu thời gian chờ quá dài hoặc âm thanh bị chia tãi tại các vị trí thiếu tự nhiên. Vì vậy, cơ sở lý thuyết của hệ thống cần bao quát cả xử lý tiếng nói, truy xuất tri thức, sinh ngôn ngữ và tổ chức luồng dữ liệu.

2.2 Nhận dạng giọng nói tự động

2.2.1 Bài toán và biểu diễn tín hiệu

Nhận dạng giọng nói tự động là bài toán ánh xạ tín hiệu âm thanh thành một chuỗi ký hiệu ngôn ngữ. Gọi $X = (x_1, x_2, \dots, x_T)$ là chuỗi đặc trưng âm học và $Y = (y_1, y_2, \dots, y_U)$ là chuỗi token đầu ra. Thông thường T lớn hơn nhiều so với U , trong khi ranh giới giữa âm thanh và

từng token không được xác định trực tiếp. Mô hình ASR vì vậy phải đồng thời học biểu diễn âm học, quan hệ ngữ cảnh và phép căn chỉnh giữa hai chuỗi có độ dài khác nhau.

Tín hiệu sóng âm thường được chia thành các khung ngắn trước khi trích xuất đặc trưng. Một biểu diễn phổ biến là log-Mel filterbank, trong đó phổ tần số được ánh xạ lên thang Mel và lấy logarithm năng lượng. Biểu diễn này làm giảm kích thước dữ liệu đầu vào, giữ lại các đặc trưng quan trọng đối với nhận dạng tiếng nói và phù hợp hơn cho mô hình học sâu so với waveform thô.

2.2.2 RNN-Transducer

Recurrent Neural Network Transducer (RNN-T) là kiến trúc nhận dạng chuỗi được thiết kế cho dữ liệu đầu vào và đầu ra chưa có căn chỉnh trực tiếp [1]. Mô hình học đồng thời quan hệ giữa các khung âm thanh và lịch sử token đã sinh. Kiến trúc gồm ba thành phần:

- **Encoder:** biến đổi chuỗi đặc trưng âm học thành biểu diễn mức cao.
- **Prediction network:** mã hóa lịch sử token đầu ra.
- **Joiner:** kết hợp biểu diễn từ encoder và prediction network để dự đoán token tiếp theo hoặc ký hiệu rỗng.

Tại mỗi bước, joiner tạo phân phối xác suất trên tập token cùng ký hiệu rỗng. Nếu ký hiệu rỗng được chọn, mô hình tiếp tục xử lý biểu diễn âm học; nếu một token được chọn, token đó được bổ sung vào chuỗi đầu ra và đưa trở lại prediction network. Nhờ kết hợp thông tin âm học với lịch sử token, RNN-T phù hợp với nhận dạng theo luồng và các ứng dụng yêu cầu độ trễ thấp.

2.2.3 Conformer và Zipformer

Conformer kết hợp self-attention của Transformer với convolution để mô hình hóa đồng thời quan hệ dài hạn và đặc trưng cục bộ trong tín hiệu tiếng nói [2]. Self-attention giúp mỗi khung âm thanh khai thác thông tin từ các vị trí xa, trong khi convolution hỗ trợ nhận biết các mẫu âm học lân cận. Tuy nhiên, việc xử lý toàn bộ chuỗi ở cùng một tần suất khung làm chi phí attention tăng khi đầu vào dài.

Zipformer là một kiến trúc encoder phát triển từ các nguyên lý của Conformer, hướng tới giảm thời gian suy luận và mức sử dụng bộ nhớ [3]. Kiến trúc sử dụng cấu trúc tương tự U-Net để xử lý chuỗi ở nhiều độ phân giải thời gian. Đặc trưng đầu vào có tần suất 100 Hz trước hết được đưa qua Conv-Embed để giảm còn 50 Hz. Sáu stack encoder tiếp theo lần lượt hoạt động ở các tần suất 50 Hz, 25 Hz, 12,5 Hz, 6,25 Hz, 12,5 Hz và 25 Hz. Các stack ở giữa xử lý chuỗi ngắn hơn nên giảm chi phí của self-attention, sau đó kết quả được tăng mẫu và kết hợp với đặc trưng ở độ phân giải cao hơn.

Zipformer block tổ chức lại các thành phần attention, convolution và feed-forward so với Conformer. Trọng số attention được tính một lần rồi tái sử dụng cho hai mô-đun self-attention và một mô-đun non-linear attention. Cơ chế tái sử dụng này giúp giảm phép tính và bộ nhớ trong khi block vẫn có thể khai thác thông tin toàn cục nhiều lần. Các kết nối bypass có trọng số học được dùng để kết hợp đầu vào với đầu ra của từng nhóm mô-đun.

Kiến trúc sử dụng BiasNorm thay cho LayerNorm nhằm duy trì thông tin về độ lớn của vector sau bước chuẩn hóa. Các hàm kích hoạt SwooshR và SwooshL được sử dụng thay cho Swish, với độ dốc không triệt tiêu hoàn toàn ở miền đầu vào âm. Những thay đổi này hỗ trợ sự ổn định của các mô-đun feed-forward và convolution trong encoder.

Trong kiến trúc nhận dạng hoàn chỉnh, Zipformer đảm nhiệm vai trò encoder âm học, còn RNN-T xác định cơ chế transducer gồm encoder, prediction network và joiner. Zipformer tạo biểu diễn từ tín hiệu tiếng nói, prediction network cung cấp ngữ cảnh token và joiner kết hợp hai nguồn thông tin để dự đoán chuỗi văn bản.

Trong đề án, mô-đun ASR sử dụng checkpoint `hynt/Zipformer-30M-RNNT-6000h`, một mô hình Zipformer-RNN-T khoảng 30 triệu tham số dành cho tiếng Việt. Theo thông tin công bố của tác giả, mô hình được huấn luyện trên khoảng 6000 giờ tiếng nói tiếng Việt tổng hợp từ nhiều bộ dữ liệu công khai và được tối ưu cho suy luận tốc độ cao [4]. Mô hình được triển khai cục bộ bằng Sherpa-ONNX, không gọi dịch vụ nhận dạng bên ngoài.

2.2.4 Các chỉ số đánh giá ASR

Word Error Rate (WER) đo tỉ lệ lỗi ở cấp từ:

$$\text{WER} = \frac{S + D + I}{N}, \quad (2.2)$$

trong đó S , D , I lần lượt là số phép thay thế, xóa và chèn; N là số từ trong câu tham chiếu. Character Error Rate (CER) được tính tương tự ở cấp ký tự và thường hữu ích với tiếng Việt khi cách tách từ chưa hoàn toàn thống nhất.

Real-Time Factor (RTF) phản ánh tốc độ xử lý:

$$\text{RTF} = \frac{T_{\text{inference}}}{T_{\text{audio}}}, \quad (2.3)$$

trong đó $T_{\text{inference}}$ là thời gian nhận dạng và T_{audio} là thời lượng âm thanh. RTF nhỏ hơn 1 cho biết mô hình xử lý nhanh hơn thời lượng thực của tín hiệu đầu vào.

2.3 Mô hình ngôn ngữ lớn

Transformer là kiến trúc nền tảng của nhiều mô hình ngôn ngữ hiện đại [5]. Thành phần cốt lõi của Transformer là self-attention, cho phép mô hình xác định mức liên quan giữa các token trong cùng một chuỗi. Với truy vấn Q , khóa K và giá trị V , scaled dot-product attention được xác định bởi:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (2.4)$$

Mô hình ngôn ngữ lớn thường sinh văn bản theo cơ chế tự hồi quy, trong đó xác suất của token tiếp theo phụ thuộc vào chuỗi token đã xuất hiện:

$$P(y_1, \dots, y_n) = \prod_{i=1}^n P(y_i \mid y_1, \dots, y_{i-1}). \quad (2.5)$$

Khả năng mô hình hóa ngữ cảnh giúp LLM tạo câu trả lời tự nhiên và tổng hợp thông tin linh hoạt. Tuy nhiên, tri thức của mô hình được lưu gián tiếp trong tham số, có thể không cập nhật hoặc không bao phủ đầy đủ một chủ đề cụ thể. Mô hình cũng có thể tạo nội dung trôi chảy nhưng không có căn cứ trong nguồn dữ liệu. Vì vậy, với hệ thống hỏi đáp theo một kho tri thức xác định, LLM cần được cung cấp ngữ cảnh bên ngoài thay vì trả lời hoàn toàn từ tri thức tham số.

Mô hình sinh câu trả lời được sử dụng trong đề án là Qwen/Qwen3-4B-Instruct-2507. Đây là mô hình ngôn ngữ nhân quả có khoảng 4 tỉ tham số, đã được hậu huấn luyện để tuân theo chỉ dẫn và hoạt động ở chế độ không sinh chuỗi suy luận ẩn [6]. Kích thước tương đối gọn cho phép mô hình được phục vụ cục bộ, trong khi khả năng tuân theo chỉ dẫn phù hợp với yêu cầu sinh câu trả lời ngắn gọn dựa trên ngữ cảnh RAG.

Trong hệ thống, Qwen không được xem là nguồn tri thức chính mà đảm nhiệm việc tổng hợp câu trả lời từ các đoạn văn bản đã truy xuất.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation kết hợp mô hình sinh với một kho tài liệu bên ngoài [7]. Thay vì đưa trực tiếp câu hỏi vào LLM, hệ thống truy xuất các đoạn văn bản liên quan và bổ sung chúng vào ngữ cảnh trước khi sinh câu trả lời. Quy trình gồm ba bước chính:

- 1) **Retrieval**: tìm các đoạn văn bản liên quan đến câu hỏi.
- 2) **Augmentation**: ghép câu hỏi, tài liệu truy xuất và chỉ dẫn thành đầu vào của LLM.
- 3) **Generation**: sinh câu trả lời dựa trên ngữ cảnh đã được cung cấp.

RAG giúp tách tri thức khỏi tham số của LLM, cho phép cập nhật kho tài liệu mà không cần huấn luyện lại mô hình sinh. Chất lượng cuối cùng phụ thuộc vào cả hai giai đoạn: retriever phải tìm được tài liệu cần thiết và generator phải sử dụng đúng thông tin trong tài liệu đó.

2.4.1 Chuẩn hóa và mở rộng truy vấn

Câu hỏi hội thoại thường ngắn, chứa từ đệm, cách gọi phổ thông hoặc lỗi nhận dạng tiếng nói. Chuẩn hóa truy vấn đưa các cách diễn đạt khác nhau về dạng thống nhất trước khi tạo embedding. Mở rộng truy vấn bổ sung các từ đồng nghĩa, thuật ngữ chuyên môn hoặc biến thể biểu đạt nhằm tăng khả năng khớp với tài liệu. Nếu mở rộng không phù hợp, truy vấn có thể bị lệch ý nghĩa và làm tăng nhiễu trong kết quả truy xuất.

Hypothetical Document Embeddings (HyDE) sử dụng mô hình ngôn ngữ để sinh một đoạn văn giả định có khả năng trả lời câu hỏi, sau đó dùng embedding của đoạn giả định này để truy xuất tài liệu thật [8].

Phương pháp có thể làm giàu truy vấn ngắn, nhưng cần thêm một lượt sinh văn bản và do đó làm tăng độ trễ. HyDE vì vậy là một lựa chọn truy xuất cần được cân nhắc theo yêu cầu của từng hệ thống, không phải thành phần bắt buộc của mọi pipeline RAG.

2.5 Truy xuất ngữ nghĩa và reranking

2.5.1 Embedding và bi-encoder

Embedding ánh xạ văn bản vào một không gian vector liên tục, trong đó các văn bản có nội dung gần nhau được kỳ vọng có vector gần nhau. Bi-encoder mã hóa truy vấn và tài liệu độc lập, nhờ đó vector tài liệu có thể được tính trước và lưu trong kho dữ liệu [9]. Khi có câu hỏi mới, hệ thống chỉ cần mã hóa câu hỏi và tìm các vector tài liệu gần nhất.

Cosine similarity là một thước đo phổ biến:

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}, \quad (2.6)$$

trong đó q và d lần lượt là vector truy vấn và tài liệu. Giá trị càng lớn thể hiện hướng của hai vector càng gần nhau.

2.5.2 Vector database và HNSW

Vector database lưu vector cùng metadata và cung cấp cơ chế tìm kiếm láng giềng gần nhất. Khi số lượng vector lớn, việc tính độ tương đồng với toàn bộ dữ liệu có chi phí tuyến tính. Approximate Nearest Neighbor Search giảm số lượng điểm cần duyệt để đổi lấy khả năng kết quả chỉ xấp xỉ láng giềng gần nhất tuyệt đối.

Hierarchical Navigable Small World (HNSW) tổ chức vector thành đồ thị nhiều tầng [10]. Tìm kiếm bắt đầu ở tầng thưa để nhanh chóng di chuyển tới vùng gần truy vấn, sau đó đi xuống các tầng dày hơn để tinh

chỉnh kết quả. HNSW thường được vector database triển khai như một cơ chế lập chỉ mục có sẵn; ứng dụng sử dụng khả năng truy vấn của hệ quản trị thay vì tự hiện thực thuật toán đồ thị.

2.5.3 Cross-encoder và truy xuất hai bước

Bi-encoder có tốc độ cao nhưng truy vấn và tài liệu được mã hóa độc lập, nên mô hình không khai thác trực tiếp tương tác giữa từng token của hai văn bản. Cross-encoder đưa đồng thời cặp truy vấn–tài liệu vào cùng một mô hình và trả về điểm liên quan. Cách làm này thường chính xác hơn nhưng phải chạy suy luận riêng cho từng cặp, dẫn tới chi phí cao hơn.

Vì vậy, nhiều hệ thống sử dụng truy xuất hai bước. Bi-encoder trước hết lấy một tập ứng viên đủ rộng để bảo đảm độ phủ. Cross-encoder sau đó đánh giá lại tập ứng viên và chọn một tập nhỏ hơn làm ngữ cảnh cho LLM. Số lượng ứng viên càng lớn có thể tăng khả năng tìm được tài liệu cần thiết, nhưng cũng làm tăng thời gian reranking. Tương tự, đưa quá nhiều tài liệu vào LLM có thể tăng độ phủ nhưng làm dài prompt và bổ sung thông tin nhiễu.

2.6 Tổng hợp giọng nói

Tổng hợp giọng nói là bài toán chuyển chuỗi văn bản thành tín hiệu âm thanh có nội dung, cách phát âm, nhịp điệu và ngữ điệu phù hợp. Một hệ thống TTS cần giải quyết quan hệ giữa đơn vị ngôn ngữ và biểu diễn âm thanh, đồng thời sinh waveform đủ nhanh cho yêu cầu tương tác. Đối với tiếng Việt, thanh điệu làm thay đổi ý nghĩa của âm tiết nên lỗi phát âm dấu, trường độ hoặc vị trí ngắt nghỉ có thể làm giảm khả năng nghe hiểu.

2.6.1 Chuẩn hóa và biểu diễn văn bản

Văn bản đầu vào có thể chứa chữ số, ký hiệu, đơn vị đo, từ viết tắt hoặc dấu định dạng không thể đọc trực tiếp một cách tự nhiên. Chuẩn hóa văn bản chuyển các biểu diễn này thành dạng phát âm, chẳng hạn chuyển số và đơn vị thành chuỗi từ tương ứng. Phonemization tiếp tục ánh xạ văn bản đã chuẩn hóa sang chuỗi đơn vị phát âm. Cách biểu diễn này giúp mô hình phân biệt cách đọc của ký tự và giảm sự phụ thuộc vào hình thức chữ viết.

Với tiếng Việt, biểu diễn phát âm cần giữ được thông tin về âm đầu, âm đệm, âm chính, âm cuối và thanh điệu. Những từ nước ngoài, tên riêng và cách viết tắt có thể không tuân theo quy tắc phát âm thông thường, do đó hệ thống thường kết hợp bộ chuyển đổi chữ–âm với từ điển phát âm cho các trường hợp đặc biệt.

2.6.2 Neural audio codec

Neural audio codec học cách nén waveform thành chuỗi mã rời rạc và khôi phục chuỗi mã đó thành âm thanh. Bộ mã hóa biến tín hiệu x thành các token âm thanh z , còn bộ giải mã tái tạo waveform:

$$z = E_{\text{codec}}(x), \quad \hat{x} = D_{\text{codec}}(z). \quad (2.7)$$

So với việc dự đoán trực tiếp từng mẫu waveform, token codec tạo ra một không gian rời rạc có độ dài ngắn hơn và có thể được mô hình hóa bằng kiến trúc ngôn ngữ. Codec phải cân bằng giữa mức nén, chất lượng tái tạo và tốc độ giải mã. Nếu mức nén quá cao, một phần thông tin về giọng nói và ngữ điệu có thể bị mất; nếu chuỗi token quá dài, chi phí sinh tự hồi quy tăng.

2.6.3 Mô hình ngôn ngữ sinh token tiếng nói

Trong hướng neural codec language model, bài toán TTS được chuyển thành bài toán sinh chuỗi token âm thanh có điều kiện theo văn bản [12].

Mô hình ngôn ngữ nhân quả dự đoán tuần tự:

$$P(z_1, \dots, z_M \mid c) = \prod_{i=1}^M P(z_i \mid z_1, \dots, z_{i-1}, c), \quad (2.8)$$

trong đó c là điều kiện đầu vào gồm biểu diễn văn bản và có thể chứa thông tin giọng nói tham chiếu; z_i là token âm thanh tại bước i . Sau khi chuỗi token được sinh, neural codec decoder chuyển chúng thành waveform.

Thông tin giọng nói tham chiếu có thể được biểu diễn bởi các token codec lấy từ một đoạn âm thanh mẫu. Khi được đưa vào điều kiện của mô hình, các token này cung cấp đặc trưng về người nói như âm sắc và phong cách phát âm. Nhờ đó, cùng một nội dung văn bản có thể được tổng hợp theo những giọng khác nhau mà không cần huấn luyện lại toàn bộ mô hình.

2.6.4 Sinh âm thanh theo luồng

Với mô hình tự hồi quy, thời gian sinh tăng theo số lượng token âm thanh. Nếu hệ thống chờ sinh xong toàn bộ chuỗi token rồi mới giải mã, thời gian tạo âm thanh đầu tiên sẽ lớn. Giải mã theo luồng khắc phục vấn đề này bằng cách gom các token mới thành từng nhóm và đưa sớm qua codec decoder.

Các nhóm token liên tiếp thường cần một vùng ngữ cảnh chồng lấn. Một số token trước đoạn hiện tại được giữ lại làm *lookback*, trong khi một số token phía sau được chờ thêm làm *lookforward*. Phần âm thanh ở vùng biên có thể được cắt hoặc chồng lấn để giảm gián đoạn giữa các lần

giải mã. Nhóm token nhỏ giúp tạo âm thanh sớm hơn nhưng làm tăng số lần gọi decoder và có thể giảm tính liền mạch; nhóm lớn giữ nhiều ngữ cảnh hơn nhưng làm tăng thời gian chờ.

Ngoài việc chia token âm thanh trong nội bộ mô hình, văn bản đầu vào cũng có thể được phân thành các đoạn trước khi tổng hợp. Đoạn dài cung cấp nhiều ngữ cảnh ngữ điệu nhưng làm tăng Time-to-First-Audio. Đoạn quá ngắn giúp phản hồi sớm nhưng dễ tạo ngắt nghỉ thiếu tự nhiên. Vì vậy, hệ thống hội thoại cần cân bằng giữa độ dài đoạn văn bản, kích thước nhóm token codec và tốc độ giải mã âm thanh.

2.7 Streaming và xử lý bất đồng bộ

Trong pipeline tuần tự, mỗi mô-đun chỉ bắt đầu sau khi mô-đun trước hoàn thành toàn bộ kết quả. Thời gian đến âm thanh đầu tiên có thể biểu diễn gần đúng:

$$T_{\text{seq}} = T_{\text{ASR}} + T_{\text{retrieval}} + T_{\text{LLM-full}} + T_{\text{TTS-first}}. \quad (2.9)$$

Streaming cho phép các mô-đun truyền kết quả từng phần. Khi LLM sinh được một đoạn văn bản đủ điều kiện, đoạn này có thể được chuyển sang TTS trong khi LLM tiếp tục sinh phần còn lại. Khi đó:

$$T_{\text{stream}} \approx T_{\text{ASR}} + T_{\text{retrieval}} + T_{\text{LLM-first-chunk}} + T_{\text{TTS-first}}. \quad (2.10)$$

Vì thời gian sinh đoạn đầu thường nhỏ hơn thời gian sinh toàn bộ câu trả lời, streaming có thể giảm đáng kể thời gian người dùng chờ âm thanh đầu tiên mà không làm giảm trực tiếp chi phí tính toán của từng mô hình.

Cơ chế này có thể được tổ chức theo mô hình producer–consumer. LLM là producer tạo các đoạn văn bản, TTS là consumer lấy từng đoạn

để tổng hợp âm thanh, còn hàng đợi bất đồng bộ tách tốc độ xử lý của hai thành phần. Nếu producer nhanh hơn consumer, dữ liệu sẽ tích lũy trong hàng đợi; nếu consumer nhanh hơn, TTS phải chờ đoạn văn bản tiếp theo. Do đó, kích thước đoạn văn bản và tốc độ của TTS cùng ảnh hưởng tới độ trễ cũng như tính liên tục của âm thanh.

2.8 Các phương pháp đánh giá

2.8.1 Đánh giá hệ thống RAG

Đánh giá RAG cần xem xét riêng chất lượng truy xuất và chất lượng câu trả lời. Một câu trả lời có thể trôi chảy nhưng không được hỗ trợ bởi tài liệu, trong khi một tập tài liệu phù hợp vẫn có thể bị LLM sử dụng không chính xác. RAGAS cung cấp các chỉ số đánh giá tự động cho quan hệ giữa câu hỏi, ngữ cảnh và câu trả lời [11]:

- **Faithfulness:** mức độ các phát biểu trong câu trả lời được hỗ trợ bởi ngữ cảnh.
- **Answer relevancy:** mức độ câu trả lời tập trung vào câu hỏi.
- **Context precision:** mức độ các tài liệu được xếp hạng cao thực sự hữu ích.
- **Context recall:** mức độ ngữ cảnh truy xuất bao phủ thông tin cần thiết để tạo câu trả lời tham chiếu.

Các chỉ số này không thay thế hoàn toàn đánh giá của con người, nhưng giúp so sánh nhiều cấu hình truy xuất theo cùng một quy trình.

2.8.2 Đánh giá độ trễ

Time-to-First-Audio (TTFA) là thời gian từ một mốc bắt đầu được xác định đến khi hệ thống tạo được phần âm thanh phản hồi đầu tiên. Với

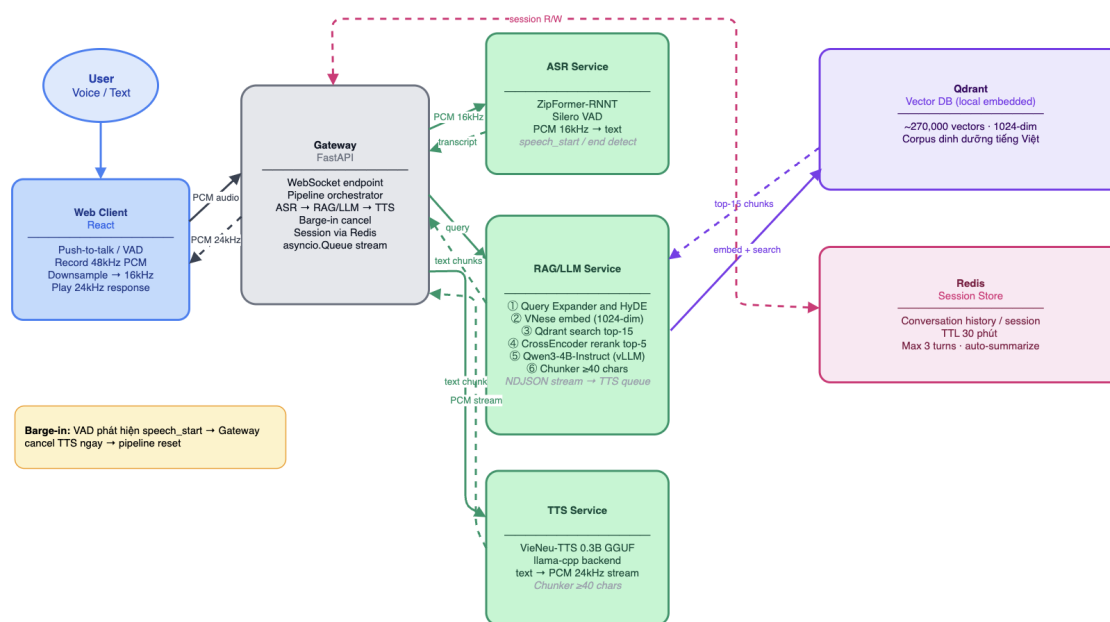
hội thoại theo lượt, mốc bắt đầu thường là thời điểm người dùng kết thúc câu hỏi. Tổng thời gian phản hồi đo thời gian để hệ thống hoàn thành toàn bộ câu trả lời, trong khi TTFA phản ánh trực tiếp khoảng im lặng mà người dùng cảm nhận trước khi bot bắt đầu nói.

Kết quả độ trễ nên được báo cáo theo phân vị thay vì chỉ dùng giá trị trung bình. Phân vị p50 biểu diễn trường hợp điển hình, còn p90 phản ánh nhóm yêu cầu chậm hơn. Việc kết hợp TTFA, tổng thời gian phản hồi, RTF của các mô-đun tiếng nói và chất lượng RAG giúp đánh giá hệ thống trên cả hai phương diện: nội dung trả lời và trải nghiệm tương tác.

CHƯƠNG 3. GIẢI PHÁP ĐỀ XUẤT

3.1 Kiến trúc tổng quan

Giải pháp được tổ chức dưới dạng pipeline mô-đun cho tương tác hai chiều bằng tiếng nói. Các thành phần chính gồm giao diện web, Gateway, mô-đun nhận dạng giọng nói, mô-đun xử lý hội thoại, mô-đun tổng hợp giọng nói, Qdrant và Redis. Gateway là điểm điều phối trung tâm, tiếp nhận dữ liệu từ trình duyệt và chuyển dữ liệu tuần tự qua các mô-đun tương ứng. Cách tổ chức này giúp phân tách rõ trách nhiệm và phạm vi xử lý của từng thành phần.



Hình 3.1: Kiến trúc tổng quan của hệ thống

Luồng xử lý chính bắt đầu khi người dùng nói qua micro. Trình duyệt thu âm, chuyển tín hiệu về PCM 16-bit và gửi tới Gateway qua WebSocket. Gateway chuyển âm thanh sang mô-đun ASR để tạo transcript tiếng Việt. Transcript được đưa vào mô-đun xử lý hội thoại để chuẩn hóa truy vấn, truy xuất tài liệu, xây dựng ngữ cảnh và sinh câu trả lời. Văn bản trả lời được chuyển từng phần sang TTS, sau đó âm thanh PCM 24 kHz được truyền ngược về trình duyệt để phát cho người dùng.

Người dùng $\rightarrow$ ASR $\rightarrow$ Xử lý hội thoại $\rightarrow$ TTS $\rightarrow$ Âm thanh phản hồi.

(3.1)

Qdrant lưu các vector biểu diễn của kho tri thức dinh dưỡng và phục vụ truy xuất ngữ nghĩa. Redis được sử dụng để lưu lịch sử hội thoại ngắn hạn theo phiên khi khả dụng. Các thành phần này hỗ trợ mô-đun xử lý hội thoại nhưng không tham gia trực tiếp vào quá trình biến đổi tín hiệu tiếng nói.

Bảng 3.1: Vai trò của các thành phần trong giải pháp

Thành phần	Vai trò
Giao diện web	Thu âm, gửi dữ liệu tiếng nói, nhận văn bản và phát âm thanh phản hồi.
Gateway	Điều phối luồng dữ liệu giữa trình duyệt và các mô-đun xử lý.
ASR	Chuyển tín hiệu tiếng nói tiếng Việt thành văn bản.
Mô-đun xử lý hội thoại	Truy xuất tri thức, xây dựng ngữ cảnh và sinh câu trả lời.
TTS	Chuyển các đoạn văn bản trả lời thành âm thanh.
Qdrant	Lưu trữ và truy vấn các vector của kho tri thức.
Redis	Lưu lịch sử hội thoại ngắn hạn và trạng thái phiên khi khả dụng.

3.2 Mô-đun xử lý tiếng nói

Mô-đun xử lý tiếng nói gồm hai chiều: nhận dạng giọng nói để tạo đầu vào văn bản và tổng hợp giọng nói để tạo đầu ra âm thanh. Hai chiều sử dụng định dạng âm thanh khác nhau phù hợp với từng mô hình, nhưng đều được Gateway điều phối trong cùng một lượt hội thoại.

3.2.1 Nhận dạng giọng nói

Âm thanh từ trình duyệt được chuyển về PCM 16-bit, đơn kênh, tần số lấy mẫu 16 kHz trước khi gửi tới ASR. Hệ thống sử dụng mô hình Zipformer–RNN-T tiếng Việt để nhận dạng transcript. VAD hỗ trợ xác định khoảng có tiếng nói và thời điểm kết thúc lượt nói; sau đó ASR xử lý đoạn âm thanh của lượt hội thoại và trả văn bản cuối cùng cho Gateway.

Transcript có thể chứa lỗi ở các thuật ngữ dinh dưỡng, từ mượn hoặc cách đọc chữ số. Chẳng hạn, “omega ba”, “can xi” hoặc “vitamin đề” có thể khác với dạng viết trong tài liệu. Vì vậy, transcript không được đưa trực tiếp vào truy xuất mà tiếp tục được chuẩn hóa tại mô-đun xử lý hội thoại.

3.2.2 Tổng hợp giọng nói

Ở chiều đầu ra, hệ thống sử dụng VieNeu-TTS để chuyển văn bản thành tín hiệu PCM 24 kHz. TTS không nhận toàn bộ câu trả lời trong một lần mà nhận các đoạn văn bản được Gateway tạo dần từ luồng sinh của LLM. Cách này cho phép bắt đầu tổng hợp âm thanh trước khi LLM hoàn thành toàn bộ câu trả lời.

Trước khi gửi sang TTS, văn bản được loại bỏ các thành phần không phù hợp với lời nói như ký hiệu Markdown, tiêu đề, dấu đầu dòng và liên kết. Việc chia đoạn ưu tiên các dấu câu và ranh giới từ để hạn chế cắt giữa một từ hoặc một cụm diễn đạt. Trình duyệt nhận các đoạn PCM và lập lịch phát nối tiếp, nhờ đó các kết quả tổng hợp riêng biệt được phát theo đúng thứ tự.

3.3 Mô-đun xử lý hội thoại

Mô-đun xử lý hội thoại biến transcript thành câu trả lời dựa trên kho tri thức. Giải pháp gồm một bước tiền xử lý và ba bước chính của RAG: Retrieval, Augmentation và Generation.

3.3.1 Tiền xử lý truy vấn

Trước khi tiền xử lý truy vấn, hệ thống áp dụng một guardrail dựa trên luật cho câu hỏi gốc. Lớp này nhận biết một số cụm từ liên quan đến tình huống khẩn cấp, yêu cầu chẩn đoán hoặc điều chỉnh thuốc và các chủ đề

rõ ràng nằm ngoài phạm vi dinh dưỡng. Nếu guardrail được kích hoạt, hệ thống trả về một thông điệp được xác định trước và bỏ qua quá trình truy xuất cũng như sinh câu trả lời bằng LLM.

Query expansion dựa trên từ điển được sử dụng để chuẩn hóa các cách diễn đạt phổ thông thành thuật ngữ thường xuất hiện trong tài liệu. Ví dụ, “tiểu đường” được mở rộng thành “đái tháo đường”, “bà bầu” thành “phụ nữ mang thai”, “omega 3” thành “axit béo omega-3”, còn “béo phì” thành “thừa cân béo phì”. Bước này không cần gọi thêm một mô hình ngôn ngữ nên có chi phí xử lý thấp.

Hệ thống cũng hỗ trợ HyDE như một cấu hình truy xuất tùy chọn. Khi được kích hoạt, Qwen sinh một đoạn văn giả định ngắn từ câu hỏi và embedding của đoạn này được dùng để tìm kiếm. HyDE có thể hỗ trợ các câu hỏi ngắn hoặc cách diễn đạt đời thường, nhưng tạo thêm một lượt suy luận; vì vậy ảnh hưởng của cơ chế này cần được xem xét cùng độ trễ.

3.3.2 Retrieval

Câu hỏi sau tiền xử lý được mã hóa bằng mô hình AITeamVN/Vietnamese_Embedding. Vector truy vấn được sử dụng để tìm kiếm trên Qdrant và lấy 15 đoạn văn bản ứng viên có độ tương đồng cao. Đây là bước truy xuất diện rộng, ưu tiên khả năng không bỏ sót tài liệu liên quan.

Các đoạn ứng viên tiếp tục được đánh giá lại bằng mô hình thanhtran/Vietnamese_Reranker. Reranker nhận từng cặp gồm câu hỏi và đoạn văn bản, sau đó sắp xếp lại theo mức độ phù hợp. Năm đoạn có điểm cao nhất được giữ lại cho bước tiếp theo. Như vậy, truy xuất được thực hiện qua hai tầng: mô hình embedding tạo tập ứng viên nhanh, còn reranker chọn tập ngữ cảnh nhỏ và chính xác hơn.

Sau reranking, hệ thống kiểm tra mức đủ tối thiểu của bằng chứng. Kết quả được chấp nhận khi có ít nhất một đoạn tài liệu và tổng nội dung truy xuất đạt tối thiểu 120 ký tự. Nếu điều kiện không đạt, hệ thống trả về thông báo chưa có đủ tài liệu thay vì gọi LLM. Trước khi xây dựng prompt, nội dung truy xuất được loại bỏ URL, nhãn nguồn và các dòng có dấu hiệu chứa chỉ dẫn nhằm can thiệp vào prompt.

3.3.3 Augmentation

Năm đoạn văn bản sau reranking được ghép thành phần ngữ cảnh của prompt. Ngoài các đoạn truy xuất, đầu vào của LLM gồm câu hỏi đã chuẩn hóa, chỉ dẫn về phạm vi tư vấn dinh dưỡng, hai ví dụ hỏi đáp định hướng cách trả lời, bản tóm tắt hội thoại và các lượt hội thoại gần nhất nếu có. Cấu trúc này bảo đảm Qwen nhận được context cụ thể để trả lời, thay vì chỉ dựa trên tri thức được lưu trong tham số mô hình.

Bảng 3.2: Thông tin được đưa vào đầu vào của mô hình ngôn ngữ

Thành phần	Mục đích
Chỉ dẫn hệ thống	Quy định phạm vi, phong cách và mức độ thận trọng của câu trả lời.
Ví dụ hỏi đáp	Định hướng cấu trúc và phong cách của câu trả lời tư vấn.
Ngữ cảnh truy xuất	Cung cấp thông tin dinh dưỡng được chọn từ kho tri thức.
Tóm tắt và lịch sử	Hỗ trợ hiểu các câu hỏi nối tiếp trong cùng phiên hội thoại.
Câu hỏi hiện tại	Xác định nội dung chính mà mô hình cần trả lời.

3.3.4 Generation

Mô hình Qwen/Qwen3-4B-Instruct-2507 được phục vụ cục bộ và nhận prompt đã bổ sung ngữ cảnh. Mô hình sinh câu trả lời theo luồng để Gateway có thể tiếp nhận từng phần văn bản ngay khi được tạo. Chỉ dẫn yêu cầu câu trả lời tập trung vào câu hỏi, bám sát ngữ cảnh truy xuất, tránh khẳng định vượt quá tài liệu và sử dụng cách diễn đạt phù hợp để chuyển sang giọng nói.

Trong khi sinh, các phần văn bản được làm sạch URL và định dạng

không phù hợp với lời nói. Sau khi hoàn thành, hệ thống kiểm tra một số phát biểu liên quan đến điều chỉnh thuốc hoặc cam kết chữa khỏi. Nếu câu trả lời chưa có câu khuyến nghị gặp bác sĩ dinh dưỡng, hệ thống bổ sung câu này vào cuối luồng. Đây là các lớp kiểm tra dựa trên luật, không phải một hệ thống kiểm định y khoa độc lập.

3.4 Chiến thuật streaming

Nếu pipeline hoạt động tuần tự, hệ thống phải chờ LLM sinh xong toàn bộ câu trả lời rồi mới bắt đầu TTS. Giải pháp đề xuất cho phép quá trình sinh văn bản và tổng hợp giọng nói chồng lấp thời gian. LLM đóng vai trò producer tạo các phần văn bản, TTS là consumer tổng hợp từng đoạn, còn Gateway duy trì hàng đợi bất đồng bộ giữa hai thành phần.

Văn bản được phân đoạn ở hai mức. Trước hết, mô-đun xử lý hội thoại tạo các phần văn bản theo dấu kết thúc câu hoặc theo ngưỡng ký tự. Gateway tiếp tục cộng dồn các phần này trong một bộ đệm trước khi gửi sang TTS. Gateway ưu tiên tách sau dấu câu khi bộ đệm đạt ít nhất 40 ký tự; nếu bộ đệm đạt 200 ký tự, toàn bộ nội dung hiện có được chuyển sang TTS để tránh chờ quá lâu. Hàng đợi văn bản gửi TTS được giới hạn ở bốn phần tử để tốc độ sinh văn bản không vượt quá xa tốc độ tổng hợp âm thanh.

Quy trình streaming gồm các bước:

- 1) Qwen sinh token theo luồng; mô-đun xử lý hội thoại gom token thành các phần văn bản.
- 2) Gateway nhận các phần văn bản, tiếp tục cộng dồn và làm sạch nội dung cho TTS.
- 3) Đoạn đủ điều kiện được đưa vào hàng đợi TTS có kích thước tối đa bốn phần tử.

- 4) VieNeu-TTS tổng hợp đoạn văn bản theo frame PCM trong khi Qwen tiếp tục sinh phần sau.
- 5) Các frame âm thanh được truyền về trình duyệt và lập lịch phát nối tiếp.

VAD còn cung cấp tín hiệu để thực hiện barge-in. Khi Silero VAD phát hiện sự kiện bắt đầu nói trong lúc bot vẫn đang trả lời, Gateway hủy tác vụ xử lý câu trả lời hiện tại, gửi yêu cầu dừng TTS và thông báo cho trình duyệt xóa bộ đệm âm thanh đang phát. Hệ thống sau đó tiếp tục thu nhận lượt nói mới. Cơ chế này giúp người dùng có thể ngắt lời bot mà không phải chờ toàn bộ câu trả lời kết thúc, đồng thời hạn chế âm thanh của lượt cũ bị phát lẫn vào lượt hội thoại mới.

Với xử lý tuần tự, thời gian đến âm thanh đầu tiên có thể biểu diễn gần đúng bởi:

$$T_{\text{seq}} = T_{\text{ASR}} + T_{\text{RAG}} + T_{\text{LLM-full}} + T_{\text{TTS-first}}. \quad (3.2)$$

Khi sử dụng streaming, hệ thống chỉ cần chờ phần văn bản đầu tiên đủ điều kiện:

$$T_{\text{stream}} = T_{\text{ASR}} + T_{\text{RAG}} + T_{\text{LLM-first-chunk}} + T_{\text{TTS-first}}. \quad (3.3)$$

Do $T_{\text{LLM-first-chunk}}$ nhỏ hơn thời gian sinh toàn bộ câu trả lời, chiến thuật này làm giảm Time-to-First-Audio mà không yêu cầu bỏ bớt bước truy xuất. Tuy nhiên, đoạn quá nhỏ có thể làm giọng đọc rời rạc và tăng số lần tổng hợp, còn đoạn quá lớn làm tăng thời gian chờ. Vì vậy, ảnh hưởng của ngưỡng chia đoạn được đánh giá thực nghiệm trong Chương 4.

CHƯƠNG 4. PHÁT TRIỂN HỆ THỐNG, THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Xây dựng kho tri thức

Kho tri thức dinh dưỡng được xây dựng từ các nguồn thông tin uy tín, sau đó trải qua các bước thu thập, làm sạch, chuẩn hóa và phân đoạn văn bản. Mục tiêu của quá trình này là tạo ra các chunk vừa đủ nhỏ để truy xuất chính xác, nhưng vẫn đủ ngữ cảnh để LLM sinh câu trả lời có ý nghĩa. Đây là bước nền tảng của hệ thống vì chất lượng RAG phụ thuộc trực tiếp vào độ tin cậy của nguồn dữ liệu và chất lượng phân đoạn văn bản.

Quy trình xử lý dữ liệu gồm các bước: loại bỏ nội dung nhiễu, chuẩn hóa unicode và khoảng trắng, tách câu hoặc đoạn theo ngữ nghĩa, tạo embedding cho từng chunk và lưu vào Qdrant. Với các tài liệu dài, chiến lược chunking cần tránh cắt giữa các ý quan trọng. Kích thước chunk là một tham số quan trọng vì ảnh hưởng đồng thời đến chất lượng truy xuất, chi phí embedding và độ chính xác của câu trả lời.

Corpus hiện tại gồm 1060 bài viết dinh dưỡng từ ba nguồn công khai có tính chuyên môn: Báo Sức khỏe và Đời sống, trang web của Viện Dinh dưỡng Quốc gia và Bệnh viện Đa khoa Quốc tế Thu Cúc. Mỗi bản ghi trong kho dữ liệu có các trường: nguồn, URL, tiêu đề, nội dung, chuyên mục và thời điểm thu thập. Nội dung được dùng để tạo kho tri thức cho các chủ đề như tăng chiều cao, chế độ ăn cho bệnh nền, vitamin, khoáng chất, dinh dưỡng cho trẻ em và phụ nữ mang thai. Sau khi thu thập, dữ liệu được làm sạch để loại bỏ nhiễu HTML, chuẩn hóa xuống dòng và ghép các đoạn rời rạc thành văn bản liền mạch.

Trong quá trình embedding, mỗi chunk được lưu cùng metadata như tiêu đề, nguồn và URL. Metadata không được đọc trực tiếp trong câu

trả lời thoại, nhưng vẫn có ích cho debug, đánh giá và truy vết nguồn dữ liệu. Khi triển khai bằng Docker, Qdrant có volume lưu trữ riêng và có cơ chế khôi phục snapshot. Điều này giúp hệ thống có thể tái lập collection mà không cần chạy lại toàn bộ pipeline thu thập, làm sạch và embedding dữ liệu.

4.2 Phát triển hệ thống

4.2.1 Thiết kế hệ thống

Thiết kế hệ thống trong giai đoạn phát triển bám theo giải pháp đã trình bày ở Chương 3. Hệ thống được tổ chức thành các khối độc lập gồm Web Client, Gateway, ASR, LLM/RAG, TTS, Qdrant và Redis. Gateway là điểm điều phối trung tâm, chịu trách nhiệm nhận audio từ client, gửi audio sang ASR, chuyển transcript sang LLM/RAG, nhận text stream từ LLM/RAG, chia văn bản thành các chunk phù hợp và gửi sang TTS để tổng hợp âm thanh.

Về công nghệ, Gateway và các worker phía sau được xây dựng bằng FastAPI để thuận tiện triển khai các luồng HTTP/WebSocket và streaming. Web Client được xây dựng bằng React để phục vụ thu âm, hiển thị trạng thái hội thoại và phát audio trả về. Qdrant được dùng làm vector database cho kho tri thức dinh dưỡng, Redis được dùng để lưu trạng thái phiên và lịch sử hội thoại ngắn hạn. Các mô hình AI được tách thành các worker riêng để thuận tiện kiểm thử, thay thế mô hình và đo latency theo từng thành phần.

Thiết kế này cho phép hệ thống chạy cục bộ trong quá trình phát triển, đồng thời có thể đóng gói bằng Docker để tái lập môi trường triển khai. Với môi trường có GPU, các thành phần nặng như ASR, embedding, reranking hoặc TTS có thể được tăng tốc bằng CUDA. Với môi trường CPU, hệ thống vẫn có thể chạy nhưng cần điều chỉnh mô hình, số luồng

xử lý và mức đồng thời phù hợp.

Luồng tương tác hai chiều bằng tiếng nói trong giai đoạn phát triển được tổ chức theo thứ tự sau:

- 1) Web Client thu audio từ người dùng, chuẩn hóa định dạng và gửi liên tục về Gateway.
- 2) Gateway chuyển audio sang ASR để tạo transcript tiếng Việt cho từng lượt nói.
- 3) Transcript được gửi sang LLM/RAG để chuẩn hóa truy vấn, truy xuất tài liệu, reranking và sinh câu trả lời dạng streaming.
- 4) Gateway gom các đoạn văn bản đủ điều kiện và chuyển từng đoạn sang TTS để tổng hợp âm thanh.
- 5) Audio từ TTS được stream ngược về Gateway và phát trên trình duyệt theo đúng session của người dùng.

Các nhóm giao tiếp quan trọng trong hệ thống gồm:

- Gateway cung cấp kênh hội thoại thời gian thực, kiểm tra trạng thái hệ thống và điều phối session.
- ASR hỗ trợ nhận dạng theo audio hoàn chỉnh và nhận dạng theo luồng có VAD.
- LLM/RAG hỗ trợ query expansion, HyDE, truy xuất hai bước, sinh câu trả lời đầy đủ, sinh câu trả lời dạng streaming và nén lịch sử hội thoại.
- TTS hỗ trợ tổng hợp audio đầy đủ, tổng hợp PCM dạng streaming và hủy tổng hợp để phục vụ barge-in.

Việc tách chế độ batch và streaming giúp quá trình phát triển thuận tiện hơn. Chế độ batch phù hợp cho kiểm thử nhanh hoặc xử lý audio

hoàn chỉnh; chế độ streaming phù hợp cho hội thoại trực tiếp. Trong đánh giá latency, các luồng streaming là trọng tâm vì chúng phản ánh trải nghiệm người dùng thực tế.

4.2.2 Xây dựng hệ thống

Quá trình xây dựng hệ thống được thực hiện theo hướng phát triển và kiểm thử từng mô-đun trước khi tích hợp end-to-end. ASR được kiểm tra với audio mẫu và audio thu trực tiếp để đánh giá khả năng tạo transcript tiếng Việt. LLM/RAG được kiểm tra bằng các câu hỏi dinh dưỡng phổ biến nhằm đánh giá query expansion, HyDE, truy xuất tài liệu, reranking và khả năng sinh câu trả lời dựa trên context. TTS được kiểm tra bằng các đoạn văn bản có độ dài khác nhau để đánh giá thời gian tạo audio đầu tiên và độ tự nhiên khi đọc.

Sau khi từng thành phần hoạt động ổn định, hệ thống được tích hợp thành pipeline hoàn chỉnh từ Web Client đến Gateway, ASR, LLM/RAG, TTS và trả audio về trình duyệt. Các kịch bản kiểm thử đơn vị tập trung vào hàm xử lý văn bản, guardrail, query expansion, lựa chọn context và các logic streaming. Các kịch bản kiểm thử tích hợp tập trung vào luồng hỏi đáp hoàn chỉnh, khả năng phát audio liên tục, barge-in và việc giữ session memory giữa các lượt hội thoại.

Hệ thống hỗ trợ chạy cục bộ và triển khai bằng Docker. Docker Compose định nghĩa các thành phần Gateway, ASR, LLM/RAG, TTS, Qdrant, Redis và Web. Gateway phụ thuộc vào ASR, LLM/RAG, TTS và Redis. Worker LLM/RAG phụ thuộc vào Qdrant và cơ chế khôi phục dữ liệu vector. Mỗi worker AI có healthcheck riêng để phát hiện lỗi khởi động. Bản cấu hình GPU override cho phép tăng tốc phần cứng bằng CUDA khi máy chủ có GPU phù hợp.

Kết quả của giai đoạn xây dựng là hệ thống có thể thực hiện một lượt

kiểm thử tương tác hai chiều bằng tiếng nói hoàn chỉnh: người dùng nói câu hỏi dinh dưỡng, hệ thống nhận dạng transcript, truy xuất tri thức, sinh câu trả lời dạng streaming, chuyển từng đoạn sang TTS và phát âm thanh phản hồi trên trình duyệt. Kết quả này là cơ sở để chuyển sang phần thực nghiệm và đánh giá độ trễ, chất lượng RAG cũng như vai trò của từng mô-đun trong pipeline.

Các tham số đáng chú ý trong hệ thống gồm:

Bảng 4.1: Một số tham số cấu hình chính

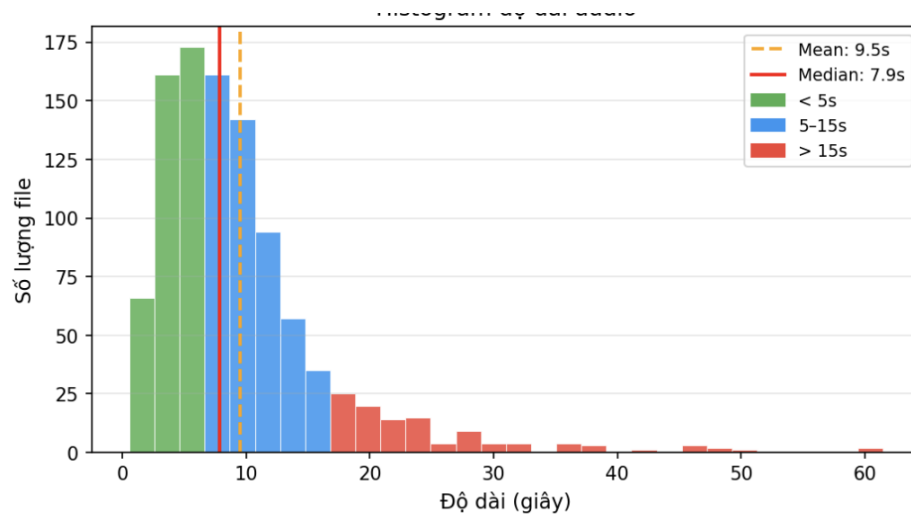
Tham số	Giá trị mặc định	Ý nghĩa
ASR_NUM_THREADS	4	Số luồng CPU cho Sherpa-ONNX.
MIN_TEXT_CHUNK_SIZE	40	Kích thước tối thiểu của text chunk trước khi gửi TTS.
Số chunk ứng viên từ Qdrant	15	Số chunk được lấy ở bước truy xuất ban đầu trước rerank.
Số chunk đưa vào LLM	5	Số chunk tốt nhất sau rerank được đưa vào đầu vào của LLM.
LLM_BACKEND	gemini	Backend sinh câu trả lời, có thể đổi sang OpenAI-compatible.
TTS_DEVICE	auto	Thiết bị chạy TTS, tự chọn CPU hoặc CUDA tùy môi trường.

4.3 Thiết lập thực nghiệm

Các thực nghiệm được thiết kế nhằm đánh giá hai khía cạnh chính của hệ thống: chất lượng câu trả lời dựa trên RAG và độ trễ trong tương tác giọng nói. Hệ thống được chạy trong môi trường Docker để tái lập cấu hình giữa các lần đo. Các thành phần Gateway, ASR, LLM/RAG, TTS, Qdrant, Redis và Web được khởi động như các service độc lập, trong đó các mô hình AI có thể sử dụng tăng tốc phần cứng bằng CUDA/GPU khi môi trường hỗ trợ.

Với đánh giá chất lượng RAG, mỗi mẫu đánh giá gồm câu hỏi, câu trả lời sinh bởi hệ thống và các ngữ cảnh được truy xuất từ kho tri thức. Các câu hỏi được chọn xoay quanh các chủ đề dinh dưỡng phổ biến như chế độ ăn cho bệnh nền, vitamin, khoáng chất, giảm cân, dinh dưỡng

cho trẻ em và phụ nữ mang thai. Với đánh giá độ trễ, hệ thống được đo trên nhiều cấu hình chunking khác nhau để phân tích ảnh hưởng của kích thước đoạn văn bản đến Time-to-First-Audio và tổng thời gian phản hồi.



Hình 4.1: Phân bố độ dài audio trong tập đánh giá

Hình trên cho thấy phần lớn các mẫu audio trong tập đánh giá có độ dài ngắn đến trung bình, tập trung chủ yếu quanh vùng dưới 15 giây. Giá trị trung bình khoảng 9.56 giây và trung vị khoảng 7.95 giây, cho thấy tập đánh giá phản ánh khá đúng kiểu câu hỏi thoại ngắn trong một hệ thống tư vấn. Tuy nhiên, phân bố vẫn có một số mẫu dài hơn 15 giây, thậm chí kéo dài tới khoảng một phút. Nhóm mẫu dài này quan trọng vì chúng giúp kiểm tra khả năng xử lý của ASR và pipeline streaming trong các trường hợp người dùng nói dài, đặt câu hỏi phức tạp hoặc mô tả nhiều điều kiện sức khỏe cùng lúc.

4.4 Đánh giá chất lượng RAG

Chất lượng của hệ thống được đánh giá bằng RAGAS, tập trung vào quan hệ giữa câu hỏi, câu trả lời và ngữ cảnh truy xuất. Các chỉ số quan trọng gồm:

- **Faithfulness:** mức độ câu trả lời bám sát ngữ cảnh được truy xuất.

- **Context Recall:** khả năng truy xuất đúng tài liệu cần thiết cho câu hỏi.
- **Answer Relevancy:** mức độ câu trả lời tập trung vào câu hỏi của người dùng.

Framework đánh giá chất lượng được xây dựng quanh RAGAS. Bộ dữ liệu đánh giá gồm câu hỏi, câu trả lời tham chiếu và ngữ cảnh liên quan. Với mỗi mẫu, hệ thống sinh câu trả lời và lưu lại context được truy xuất. Sau đó, RAGAS tính các chỉ số như faithfulness, answer relevancy, context precision và context recall. Cách đánh giá này phù hợp với hệ thống RAG vì nó không chỉ kiểm tra câu trả lời cuối cùng mà còn đánh giá chất lượng truy xuất ngữ cảnh.

Bảng dưới đây trình bày kết quả đánh giá RAGAS trên 360 mẫu với mô hình đánh giá Qwen/Qwen3.5-27B. Ba cấu hình được so sánh theo số lượng chunk ứng viên ban đầu và số chunk tốt nhất được đưa vào đầu vào của LLM.

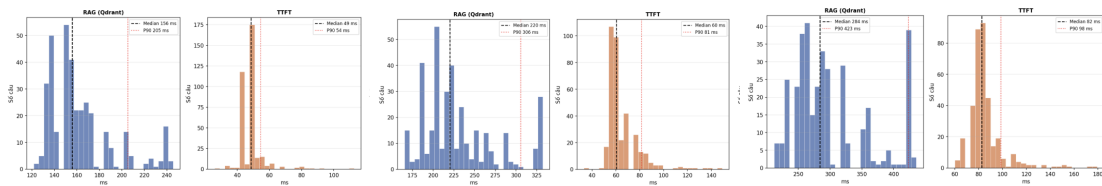
Bảng 4.2: Kết quả đánh giá chất lượng RAG theo cấu hình truy xuất

Cấu hình truy xuất	Answer Correctness	Faithfulness	Context Recall	Context Precision
10 chunk ứng viên, chọn 3 chunk	0.5887	0.7375	0.5184	0.5426
15 chunk ứng viên, chọn 5 chunk	0.6098	0.7723	0.6239	0.5709
20 chunk ứng viên, chọn 7 chunk	0.6136	0.7897	0.6830	0.5493

Kết quả cho thấy khi tăng số lượng chunk ứng viên và số chunk được đưa vào LLM, context recall và faithfulness có xu hướng tăng. Tuy nhiên, context precision không tăng đơn điệu: cấu hình 20 chunk ứng viên, chọn 7 chunk có recall cao hơn nhưng precision thấp hơn cấu hình 15 chunk ứng viên, chọn 5 chunk. Điều này cho thấy việc đưa thêm ngữ cảnh không luôn làm câu trả lời tốt hơn; hệ thống cần cân bằng giữa độ phủ

tài liệu, độ nhiều của context và chi phí latency. Trong phạm vi đồ án, cấu hình 15 chunk ứng viên, chọn 5 chunk được xem là điểm cân bằng hợp lý giữa chất lượng truy xuất và độ trễ cho voicebot.

Phân phân tích ảnh hưởng của tham số tập trung vào kích thước đoạn văn bản được chuyển từ LLM sang TTS, vì tham số này tác động trực tiếp đến độ trễ phản hồi và trải nghiệm nghe. Các cấu hình không chia đoạn, 80 ký tự, 40 ký tự và 20 ký tự được so sánh theo Time-to-First-Audio, tổng thời gian phản hồi và nguy cơ ngắt nhịp khi tổng hợp giọng nói.



Hình 4.2: Phân bố thời gian RAG và thời gian sinh phần trả lời đầu tiên theo các cấu hình truy xuất

Ba biểu đồ trên minh họa ảnh hưởng của cấu hình truy xuất đến thời gian xử lý của khối LLM/RAG. Mỗi cấu hình gồm hai nhóm thời gian chính: thời gian RAG, tức thời gian mở rộng truy vấn, truy xuất và reranking; và thời gian sinh phần trả lời đầu tiên của LLM. Khi tăng số lượng chunk ứng viên và số chunk đưa vào LLM, thời gian RAG có xu hướng tăng vì hệ thống phải xử lý nhiều ứng viên hơn ở bước reranking và đưa nhiều ngữ cảnh hơn vào prompt. Ngược lại, thời gian sinh phần trả lời đầu tiên của LLM biến động ít hơn, do phần này phụ thuộc nhiều vào backend LLM và độ dài prompt sau khi đã chọn context.

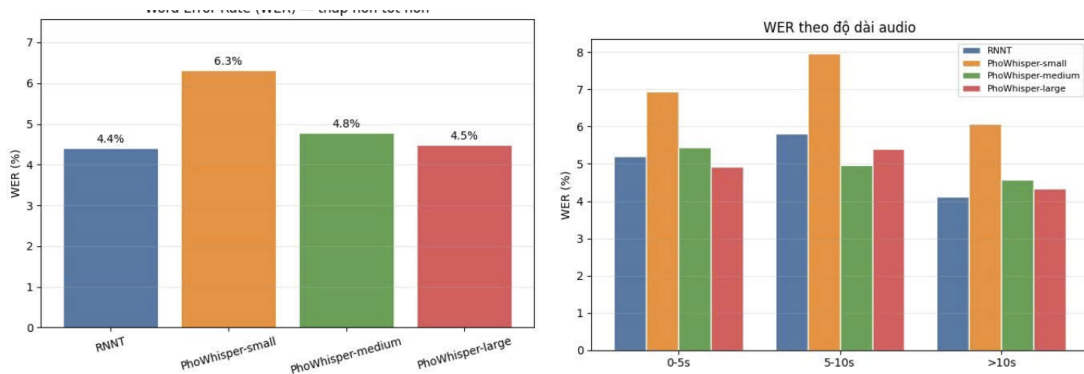
Các biểu đồ này giúp giải thích lý do đồ án không chỉ chọn cấu hình có nhiều tài liệu nhất. Cấu hình truy xuất lớn hơn có thể cải thiện độ phủ ngữ cảnh, nhưng cũng làm tăng độ trễ trước khi LLM bắt đầu sinh câu trả lời. Vì voicebot cần phản hồi nhanh, cấu hình truy xuất phải cân bằng giữa chất lượng RAG và độ trễ cảm nhận của người dùng.

Bên cạnh chất lượng RAG, đồ án còn đánh giá các thành phần riêng lẻ. ASR có thể được đo bằng WER và CER khi có transcript chuẩn. TTS

được đánh giá bằng real-time factor, thời gian tạo chunk đầu tiên và đánh giá chủ quan MOS. Pipeline tổng thể được đo bằng thời gian từ lúc kết thúc câu nói đến audio đầu tiên. Việc tách các chỉ số theo module giúp xác định thành phần nào là nút thắt trong từng cấu hình.

4.5 Đánh giá độ trễ

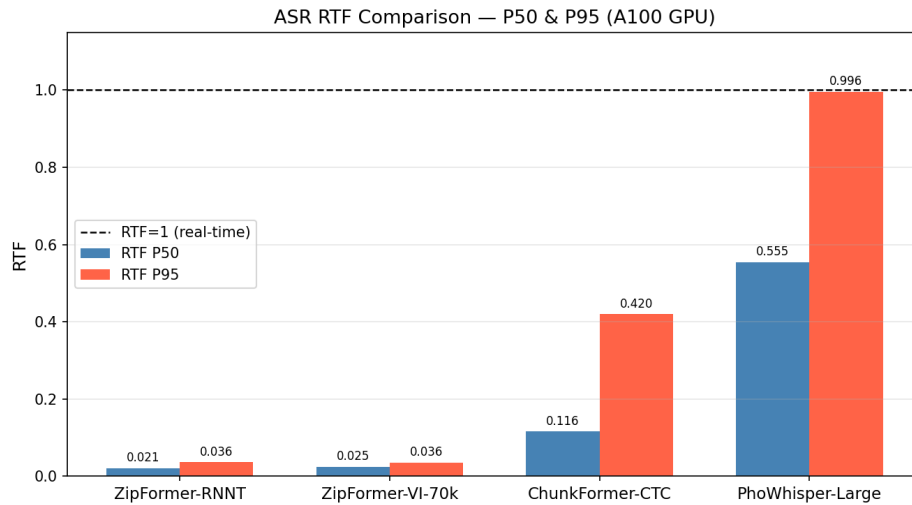
Độ trễ là tiêu chí cốt lõi của một hệ thống voicebot thời gian thực. Đồ án tập trung đo các thành phần chính: thời gian ASR sinh transcript, thời gian RAG truy xuất và rerank, thời gian LLM sinh token đầu tiên, thời gian TTS tạo audio đầu tiên và thời gian end-to-end từ lúc người dùng nói xong đến lúc nghe phản hồi.



Hình 4.3: So sánh WER của các mô hình ASR theo mô hình và theo độ dài audio

Hình trên so sánh lỗi nhận dạng tiếng nói của nhiều mô hình ASR trên tập đánh giá tiếng Việt. Kết quả tổng thể cho thấy RNNT đạt WER khoảng 4.4%, PhoWhisper-small khoảng 6.3%, PhoWhisper-medium khoảng 4.8% và PhoWhisper-large khoảng 4.5%. Khi phân tích theo độ dài audio, WER thay đổi giữa các nhóm 0–5 giây, 5–10 giây và trên 10 giây, cho thấy chất lượng ASR không chỉ phụ thuộc vào mô hình mà còn phụ thuộc vào đặc điểm câu nói. Điều này phản ánh đặc điểm quan trọng của pipeline voicebot: transcript đầu vào không phải lúc nào cũng hoàn hảo, và lỗi ASR có thể lan truyền sang bước truy xuất tri thức. Vì vậy, hệ thống cần có các bước giảm tác động của lỗi nhận dạng, chẳng hạn

chuẩn hóa thuật ngữ dinh dưỡng, query expansion và truy xuất dựa trên nhiều biến thể câu hỏi.



Hình 4.4: So sánh real-time factor của các mô hình ASR

Biểu đồ real-time factor cho thấy sự khác biệt rõ giữa các mô hình ASR về tốc độ xử lý. Đường tham chiếu RTF bằng 1 biểu thị ngưỡng real-time: nếu RTF nhỏ hơn 1, mô hình xử lý nhanh hơn thời lượng audio đầu vào. ZipFormer-RNNT và ZipFormer-VI-70k có RTF rất thấp ở cả p50 và p95, cho thấy khả năng xử lý nhanh và ổn định hơn cho bài toán hội thoại thời gian thực. ChunkFormer-CTC vẫn nằm dưới ngưỡng real-time nhưng có p95 cao hơn, nghĩa là trong một số trường hợp chậm hệ thống có thể mất nhiều thời gian hơn. PhoWhisper-Large có p95 gần 1, tức là các trường hợp chậm gần sát ngưỡng real-time. Kết quả này cho thấy lựa chọn mô hình ASR ảnh hưởng trực tiếp đến độ trễ đầu vào của toàn bộ pipeline.

Các chỉ số được báo cáo theo percentile p50 và p90 để phản ánh trải nghiệm điển hình và các trường hợp phản hồi chậm. Trong đó, Time-to-First-Audio là chỉ số đặc biệt quan trọng vì quyết định cảm nhận phản hồi tức thời của người dùng. Cơ chế streaming song song giữa LLM và TTS được kỳ vọng giúp giảm đáng kể chỉ số này so với pipeline tuần tự.

Thí nghiệm độ trễ được thực hiện trên bốn cấu hình: không chunking,

chunk 80, chunk 40 và chunk 20. Mỗi cấu hình gồm 250 lượt đo. Kết quả thống kê cho thấy chunking làm giảm mạnh Time-to-First-Audio so với pipeline full. Bảng dưới đây tóm tắt một số kết quả chính:

Bảng 4.3: Kết quả đánh giá độ trễ theo cấu hình chunk

Cấu hình	Số mẫu	TTFA mean	TTFA p50	TTFA p90
Full	250	1614.2 ms	1523.0 ms	1760.9 ms
Chunk 80	250	1059.5 ms	1055.1 ms	1179.4 ms
Chunk 40	250	748.8 ms	746.7 ms	821.6 ms
Chunk 20	250	603.9 ms	603.4 ms	660.0 ms

Kết quả này cho thấy chiến thuật streaming có tác động rõ rệt đến độ trễ cảm nhận. Cấu hình full có p50 khoảng 1523 ms, gần sát ngưỡng mục tiêu 1.5 giây và có p90 vượt ngưỡng. Khi giảm kích thước chunk xuống 40 ký tự, p50 TTFA giảm còn khoảng 747 ms và p90 còn khoảng 822 ms. Với chunk 20, TTFA tiếp tục giảm nhưng có nguy cơ làm giọng nói bị ngắt đoạn nhiều hơn. Vì vậy, chunk 40 là lựa chọn cân bằng hơn giữa độ trễ và độ tự nhiên của audio.

Ngoài TTFA, tổng thời gian xử lý cũng giảm khi dùng chunking. Trung bình cấu hình full cần khoảng 4333.5 ms, trong khi chunk 40 cần khoảng 1508.9 ms và chunk 20 cần khoảng 1258.8 ms. Điều này phản ánh lợi ích của việc chồng lấp thời gian sinh văn bản và thời gian tổng hợp giọng nói. Tuy nhiên, kết quả latency cần được phân tích cùng với đánh giá MOS hoặc nghe thử để tránh tối ưu quá mức theo số liệu mà làm giảm trải nghiệm âm thanh.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã đề xuất và xây dựng một hệ thống tư vấn dinh dưỡng thời gian thực qua giọng nói dựa trên pipeline mô-đun, streaming bất đồng bộ và RAG. Hệ thống kết hợp các mô-đun ASR, LLM và TTS để tạo thành pipeline tương tác hai chiều bằng tiếng nói hoàn chỉnh. Điểm trọng tâm của giải pháp là tối ưu Voice RAG dưới ràng buộc độ trễ: câu hỏi được kiểm tra an toàn và chuẩn hóa trước khi truy xuất, tài liệu được chọn theo mức phù hợp, câu trả lời được sinh dựa trên ngữ cảnh truy xuất, và luồng LLM–TTS được điều phối để giảm thời gian phản hồi âm thanh đầu tiên.

Các đóng góp chính của đồ án gồm: thiết kế pipeline mô-đun cho hệ thống tương tác hai chiều bằng tiếng nói trong chủ đề dinh dưỡng tiếng Việt; tổ chức pipeline Voice RAG dưới ràng buộc độ trễ của kênh thoại; áp dụng query expansion, truy xuất tài liệu và reranking để cải thiện chất lượng bằng chứng; tổ chức chiến thuật phân đoạn văn bản dựa trên dấu câu và ranh giới từ kết hợp streaming song song giữa LLM và TTS; bổ sung guardrail an toàn cho các tình huống nhạy cảm; xây dựng bộ tiêu chí đánh giá kết hợp giữa chất lượng RAG, Time-to-First-Audio và độ trễ end-to-end.

Về mặt hiện thực, đồ án đã xây dựng được mã nguồn có đầy đủ các lớp cần thiết cho hệ thống: frontend React để thu và phát audio; Gateway FastAPI để điều phối WebSocket; ASR worker cho nhận dạng tiếng nói; LLM worker cho RAG và streaming câu trả lời; TTS worker cho tổng hợp giọng nói; Docker Compose để triển khai nhiều dịch vụ; các script đánh giá RAGAS, ASR, TTS và latency. Các thành phần này tạo thành một nền tảng thử nghiệm hoàn chỉnh cho bài toán tối ưu voicebot theo cả chất lượng và độ trễ.

Kết quả đánh giá chunking cho thấy kiến trúc streaming song song là hướng đi hiệu quả. Khi chuyển từ pipeline full sang chunk 40, p50 Time-to-First-Audio giảm từ khoảng 1523 ms xuống khoảng 747 ms. Điều này chứng minh rằng không cần thay đổi mô hình nền, chỉ cần tổ chức lại luồng xử lý, kiểm soát kích thước chunk và chồng lấp LLM với TTS cũng có thể cải thiện đáng kể trải nghiệm hội thoại.

5.2 Hạn chế

Hệ thống vẫn tồn tại một số hạn chế. Chất lượng nhận dạng giọng nói có thể giảm trong môi trường nhiễu hoặc khi người dùng nói quá nhanh. Kho tri thức dinh dưỡng cần tiếp tục được mở rộng và cập nhật để bao phủ nhiều tình huống hơn. Ngoài ra, pipeline nhiều mô-đun tuy linh hoạt nhưng vẫn có chi phí vận hành và độ phức tạp triển khai cao hơn so với một mô hình end-to-end.

Một hạn chế khác là chất lượng cuối cùng phụ thuộc mạnh vào dữ liệu corpus. Nếu câu hỏi nằm ngoài phạm vi các bài viết đã thu thập, RAG có thể truy xuất ngữ cảnh không đủ thông tin, khiến LLM phải dựa nhiều hơn vào kiến thức nền. Bên cạnh đó, các mô hình embedding và reranking có chi phí tính toán đáng kể, đặc biệt khi chạy CPU. TTS cũng có trade-off giữa thời gian tạo audio đầu tiên và độ mượt khi đọc: chunk nhỏ giúp phản hồi nhanh hơn nhưng có thể tạo cảm giác ngắt câu.

Trong triển khai hiện tại, một số tham số vẫn cần tuning theo môi trường chạy thực tế, ví dụ số chunk ứng viên lấy từ Qdrant, số chunk tốt nhất đưa vào LLM, kích thước chunk văn bản cho TTS, backend LLM, thiết bị chạy ASR và TTS, cũng như mức concurrency. Vì vậy, kết quả trên một máy cụ thể không nên được xem là kết quả tuyệt đối cho mọi môi trường triển khai.

5.3 Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo các hướng sau:

- 1) Tiếp tục mở rộng kho tri thức với nhiều nguồn y tế và dinh dưỡng được kiểm chứng hơn.
- 2) Hoàn thiện query triage để nhận biết câu hỏi ngoài phạm vi dinh dưỡng, câu hỏi mơ hồ và các tình huống cần khuyến nghị gặp bác sĩ.
- 3) Mở rộng truy xuất bằng hybrid retrieval, context compression nhẹ và semantic cache cho các câu hỏi phổ biến.
- 4) Cải thiện khả năng xử lý đồng thời bằng cách đo real-time factor của TTS, giới hạn số yêu cầu tổng hợp và hủy phản hồi cũ theo session.
- 5) Cải thiện khả năng xử lý tiếng nói trong môi trường nhiễu bằng VAD và ASR streaming tốt hơn.
- 6) Cá nhân hóa tư vấn dựa trên lịch sử hội thoại, độ tuổi, bệnh nền và mục tiêu dinh dưỡng của người dùng.

Ngoài các hướng trên, hệ thống có thể được mở rộng theo hướng đánh giá liên tục trong quá trình vận hành. Mỗi phiên hội thoại có thể ghi lại transcript, context truy xuất, câu trả lời, thời gian từng module và phản hồi người dùng. Dữ liệu này giúp phát hiện các câu hỏi corpus gap, bổ sung tài liệu mới vào kho tri thức và cải thiện từ điển query expansion. Khi số lượng dữ liệu thực tế đủ lớn, hệ thống có thể chuyển từ tuning thủ công sang tối ưu dựa trên thống kê sử dụng.

Tài liệu tham khảo

- [1] Alex Graves. Sequence Transduction with Recurrent Neural Networks. arXiv:1211.3711, 2012. <https://arxiv.org/abs/1211.3711>
- [2] Anmol Gulati et al. Conformer: Convolution-augmented Transformer for Speech Recognition. Interspeech, 2020. <https://arxiv.org/abs/2005.08100>
- [3] Zengwei Yao et al. Zipformer: A Faster and Better Encoder for Automatic Speech Recognition. ICLR, 2024. <https://arxiv.org/abs/2310.11230>
- [4] hynt. Vietnamese Speech-to-Text (ASR) – ZipFormer-30M-RNNT-6000h. Hugging Face model card. <https://huggingface.co/hynt/Zipformer-30M-RNNT-6000h>
- [5] Ashish Vaswani et al. Attention Is All You Need. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- [6] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [7] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. <https://arxiv.org/abs/2005.11401>
- [8] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels. ACL, 2023. <https://arxiv.org/abs/2212.10496>
- [9] Nils Reimers, Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019. <https://arxiv.org/abs/1908.10084>

- [10] Yu. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs. IEEE TPAMI, 2020. <https://arxiv.org/abs/1603.09320>
- [11] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217, 2023. <https://arxiv.org/abs/2309.15217>
- [12] Chengyi Wang et al. Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers. arXiv:2301.02111, 2023. <https://arxiv.org/abs/2301.02111>